Développement MVP CongoLogis RDC

Initialisation et Architecture Technique

- Configuration de l'environnement de développement
  - Installer les outils de développement (Python, Node.js, PostgreSQL).
  - Configurer les environnements virtuels pour Python et Node.js.
  - Cloner les dépôts Git pour le backend et le frontend.
- Mise en place des dépôts Git
  - Créer un dépôt Git privé pour le backend (Django).
  - Créer un dépôt Git privé pour le frontend (React PWA).
  - Configurer les hooks Git de base (si nécessaire).
- Structure de projet Backend (Django)
  - Initialiser le projet Django et les applications nécessaires (ex: users, properties, tenants, payments).
  - Configurer les paramètres de base de Django (base de données, fuseau horaire, langues).
  - Mettre en place Django REST Framework.
- Structure de projet Frontend (React PWA)
  - Initialiser l'application React avec Create React App ou Vite (pour PWA).
  - Configurer Redux Toolkit pour la gestion de l'état global.
  - Intégrer Axios pour les requêtes HTTP.
  - Configurer Tailwind CSS pour le style.
- Configuration de la base de données PostgreSQL
  - Créer la base de données PostgreSQL pour le projet.
  - Configurer les accès à la base de données dans les paramètres Django.
  - Appliquer les migrations initiales de Django.

Gestion des Utilisateurs et Biens

- Création de compte bailleur
  - Développer le formulaire d'inscription (email, mot de passe, téléphone) sur le frontend.
  - Implémenter l'API d'enregistrement du bailleur sur le backend.
  - Ajouter la validation des données d'entrée (format email, force du mot de passe, format téléphone).
  - Implémenter la gestion de session/token d'authentification après inscription.
- Gestion des biens immobiliers
  - Développer le formulaire d'ajout de bien immobilier (adresse, type, description) sur le frontend.
  - Implémenter l'API CRUD (Create, Read, Update, Delete) pour les biens immobiliers sur le backend.
  - Développer l'interface utilisateur pour afficher la liste des biens associés au bailleur connecté.
  - Implémenter la sélection d'un bien immobilier pour y associer un locataire.
- Gestion des locataires
  - Développer le formulaire d'ajout de locataire (nom, tel, adresse, montant loyer, date début bail, caution, fréquence paiement) sur le frontend.
  - Implémenter l'API CRUD pour les locataires sur le backend.
  - Développer la fonctionnalité d'association d'un locataire à un bien immobilier spécifique.
  - Développer l'interface utilisateur pour afficher la liste des locataires gérés par le bailleur.
- Historique des contrats
  - Créer le modèle de données Contrat pour stocker les informations clés de chaque bail (dates, montants).
  - Implémenter l'API pour récupérer l'historique des contrats d'un locataire donné.
  - Développer l'interface utilisateur pour visualiser et filtrer l'historique des contrats d'un locataire.

Intégration des Paiements Mobile Money

- Recherche et analyse des APIs Mobile Money
  - Identifier les APIs spécifiques de M-Pesa, Orange Money, Airtel Money disponibles pour les intégrations de paiement en RDC.
  - Évaluer la documentation technique, les exigences de sécurité et les SDKs fournis par chaque opérateur.
  - Obtenir les accès aux environnements de test (sandbox) pour au moins un opérateur (ex: M-Pesa).
- Intégration API de paiement (backend)
  - Implémenter l'intégration initiale avec l'API du premier opérateur Mobile Money choisi (ex: M-Pesa).
  - Développer l'API backend pour initier une demande de paiement depuis CongoLogis vers le numéro du locataire.
  - Mettre en place un webhook ou un endpoint spécifique pour recevoir les notifications de confirmation de paiement de l'opérateur.
  - Gérer les différents statuts de transaction (en attente, confirmé, échoué) via un système de logs et de mises à jour de statut.
- Flux de paiement utilisateur (frontend)
  - Créer une interface conviviale pour le bailleur afin d'initier une demande de paiement pour un locataire spécifique.
  - Afficher le statut en temps réel du paiement (en attente, confirmé, échoué) au bailleur après l'initiation.
  - Mettre en place des notifications frontend pour informer le bailleur de l'évolution du paiement.
- Stockage des données de paiement
  - Créer le modèle de données TransactionPaiement pour enregistrer le montant, le numéro de transaction Mobile Money, l'opérateur, la date et le statut.
  - Implémenter la logique d'enregistrement automatique de chaque paiement après confirmation par l'opérateur Mobile Money.
  - Assurer la liaison entre la transaction et le locataire/bien immobilier concerné.

Génération et Gestion des Reçus Numériques

- Génération de reçus PDF
  - Intégrer une bibliothèque de génération de PDF côté backend (ex: ReportLab ou WeasyPrint pour Django).
  - Définir le template détaillé du reçu PDF incluant le nom du bailleur, du locataire, le montant payé, le mois concerné, la date et le numéro de transaction.
  - Implémenter la génération automatique d'un reçu au format PDF dès qu'un paiement est confirmé.
- Fonctionnalité QR Code
  - Intégrer une bibliothèque de génération de QR Code (ex: qrcode pour Python).
  - Générer un QR Code unique pour chaque reçu, encodant une URL de vérification spécifique au reçu.
- Vérification des reçus (backend)
  - Développer un endpoint public et sécurisé sur le backend permettant de vérifier l'authenticité d'un reçu via son QR Code.
  - Cet endpoint doit afficher les informations clés du reçu et confirmer sa validité.
- Accès et partage des reçus
  - Implémenter la fonctionnalité de téléchargement du reçu PDF depuis l'interface utilisateur.
  - Intégrer une option de partage du reçu (lien ou fichier PDF) via WhatsApp ou d'autres plateformes.
  - Développer une section dans l'interface pour visualiser l'historique de tous les reçus générés pour chaque locataire.

Mode Hors-ligne (Offline-first)

- Configuration PWA (Frontend)
  - Configurer l'application React en Progressive Web App (PWA) pour une expérience hors ligne.
  - Créer et configurer le fichier manifest.json avec les informations de l'application et les icônes.
- Implémentation des Service Workers
  - Enregistrer un Service Worker pour intercepter les requêtes réseau et gérer le cache.
  - Implémenter une stratégie de cache pour les assets statiques de l'application (HTML, CSS, JS, images).
  - Développer la logique de cache pour les données essentielles des locataires et biens immobiliers.
  - Mettre en place une "file d'attente" pour les actions utilisateur effectuées hors ligne (ex: initiation de paiement, ajout de locataire).
- Utilisation d'IndexedDB
  - Configurer IndexedDB pour stocker temporairement les données des paiements et autres actions effectuées hors ligne.
  - Développer la logique de synchronisation automatique des données stockées dans IndexedDB vers le backend dès que la connexion Internet est rétablie.
- Support Backend pour le mode hors-ligne
  - Modifier les APIs REST existantes pour les rendre tolérantes aux doublons (idempotence) via des identifiants uniques côté client.
  - Créer un endpoint de synchronisation batch sur le backend pour recevoir et traiter plusieurs actions hors ligne en une seule requête.
  - Implémenter la gestion des IDs temporaires générés côté client pour éviter les conflits lors de la synchronisation.

Monétisation (Freemium & Commission)

- Modèle Freemium (Backend)
  - Ajouter un champ plan_type (ex: 'Gratuit', 'Premium') au modèle Bailleur dans la base de données.
  - Implémenter un middleware Django pour intercepter les requêtes d'ajout de locataire.
  - Développer la logique de vérification du nombre de locataires pour les comptes "Gratuit" et bloquer l'ajout au-delà de 5.
  - Créer une API pour permettre au bailleur de changer son plan_type (passer au Premium).
- Intégration du système de paiement d'abonnement
  - Rechercher et intégrer une API de paiement pour les abonnements (ex: Stripe ou une passerelle Mobile Money locale si disponible pour abonnements).
  - Développer l'interface utilisateur pour la souscription à l'abonnement Premium, affichant les avantages.
  - Implémenter la logique de traitement des paiements récurrents pour les abonnements.
- Commission de 1 % par transaction
  - Modifier la logique de traitement des paiements Mobile Money pour calculer et déduire automatiquement 1 % du montant du loyer.
  - Créer un modèle PlatformFee pour enregistrer chaque transaction de commission (montant, date, transaction liée).
  - Assurer que le montant net crédité au bailleur est le montant du loyer moins la commission.

Outils et Infrastructures Support

- Configuration de Celery et Redis
  - Installer et configurer Celery comme système de file d'attente de tâches asynchrones.
  - Installer et configurer Redis comme broker pour Celery et cache.
  - Intégrer Celery pour les futures tâches d'envoi de SMS et de rappels.
  - Adapter la génération de reçus PDF pour s'exécuter de manière asynchrone via Celery.
- Configuration de l'environnement de production
  - Mettre en place un environnement de staging (pré-production) pour les tests finaux.
  - Configurer un pipeline d'intégration continue/déploiement continu (CI/CD) de base (ex: GitHub Actions, GitLab CI).
  - Déployer l'application backend et frontend sur un serveur de production (ex: AWS, Google Cloud, Heroku).

Presented with xmind